**1. Problem Title & Source**

Title: **Rock Sample Range**

Source: **TCS NQT Previous Coding Questions**

**2. Problem Statement (Clear Version)**

A geologist is studying **rock samples collected from different locations**.

Each rock sample is assigned a number representing its **mineral concentration level**.

You are given:

- N → number of rock samples
- arr[i] → mineral concentration value of each rock sample

The geologist then performs **Q queries**.

Each query consists of two integers:

L R

For each query, you must determine **how many rock samples have values within the range [L, R] (inclusive)**.

In simple terms:

- Count how many elements in the array satisfy:

L ≤ arr[i] ≤ R

for every query.

**3. Examples**

**Example 1**

Input

5

1 3 5 7 9

2

2 6

4 10

Output

2

3

Explanation

Rock samples

[1, 3, 5, 7, 9]

Query 1

2 ≤ value ≤ 6

Values

3, 5

Count

2

Query 2

4 ≤ value ≤ 10

Values

5, 7, 9

Count

3

**Example 2**

Input

6

2 4 6 8 10 12

1

5 11

Output

3

Explanation

Numbers in range

6, 8, 10

Count

3

**4. Constraints**

1 ≤ N ≤ 1000

1 ≤ Q ≤ 100

1 ≤ arr[i] ≤ 10^5

Observations

- Each query asks for **count of numbers in a range**
- Direct scanning works due to small constraints

**5. Core Concept (Pattern)**

**Range Query / Counting**

Basic idea:

Check each element for every query

**6. Thought Process**

**Step 1 — Read Input**

We receive

N → number of samples

array of mineral values

Q → number of queries

**Step 2 — Process Each Query**

For each query:

L R

Traverse array and count:

```
if L ≤ arr[i] ≤ R
   count++
```

**Step 3 — Print Result**

Print count for every query.

**7. Visualization**

Example

arr = [1,3,5,7,9]

Query

L = 2

R = 6

Check values

| Value | In Range |
|---|---|
| 1 | No |
| 3 | Yes |
| 5 | Yes |
| 7 | No |
| 9 | No |

Count

2

**8. Pseudocode**

```
read N
read array
```

```
read Q

repeat Q times

   read L R

   count = 0

   for each element in array

      if L ≤ element ≤ R
         count++

   print count
```

**9. Code Implementation**

**Python**

```python
n = int(input())
arr = list(map(int,input().split()))

q = int(input())

for _ in range(q):

   l, r = map(int,input().split())

   count = 0

   for x in arr:
      if l <= x <= r:
         count += 1

   print(count)
```

**Java**

```java
import java.util.*;
```

```
class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];

        for(int i=0;i<n;i++)
            arr[i] = sc.nextInt();

        int q = sc.nextInt();

        while(q-- > 0){

            int l = sc.nextInt();
            int r = sc.nextInt();

            int count = 0;

            for(int x : arr){

                if(x >= l && x <= r)
                    count++;
            }

            System.out.println(count);
        }
    }
}
```

**10. Time & Space Complexity**

**Time Complexity**

O(N × Q)

Where

N = number of samples

Q = number of queries

**Space Complexity**

O(1)

Only counters used.

**11. Common Mistakes**

1. Using < instead of ≤
2. Forgetting to reset counter
3. Incorrect input reading
4. Using unnecessary sorting

**12. Detailed Dry Run**

Input

arr = [1,3,5,7,9]

query = 4 10

Check

| Value | Condition |
|---|---|
| 1 | No |
| 3 | No |
| 5 | Yes |
| 7 | Yes |
| 9 | Yes |

Count

3

**13. Real-Life Applications**

Range queries appear in:

- Database filtering
- Sensor monitoring systems
- Financial data analysis
- Inventory range checks

**14. Common Traps**

1. Using wrong range condition

2. Not resetting count
3. Misreading input order
4. Printing result incorrectly

**15. Related Problems**

Range query problems:

LC 303 Range Sum Query

LC 307 Range Sum Query Mutable

LC 2080 Range Frequency Queries

**16. Alternate Approaches**

| Approach | Complexity | Notes |
|---|---|---|
| Direct scan | O(NQ) | simple |
| Sorting + binary search | O(Q log N) | faster for large inputs |

**17. Why This Solution Works**

Each query independently checks how many values fall within the range.

Since the dataset size is small, scanning the array for each query is efficient and simple.

**18. Variations / Follow-Up Questions**

Possible interview variations:

1. Return **sum of elements in range**
2. Find **maximum value in range**
3. Use **prefix sum optimization**
4. Handle **large N and Q efficiently**